Name: Himansh Purswani

Title: SOC Theoretical Fundamentals and Practical Operations

1. SOC Fundamentals and operations:

- In this task it was required to study theoretical knowledge about **SOC** which is the **mission controller** of the **organization's cybersecurity**. its main goal is to reduce the time an **attacker** spends in the network.
- Then to Study the 3 tier analysts of SOC, the SOC manager and the threat hunters; Tier 1 is the first line of defence. They monitor SIEM, filter out the false positives, and escalate real threats, Tier 2 take a deep dive into escalated alerts. They perform forensics, contain the threat, and determine the "how" and "why" of the threat; Tier 3 analysts look for threats that haven't triggered alerts yet. They use advanced threat intelligence and then lastly the SOC manager over sees the operation, manages the budget, and ensures the team meets the performance targets.
- Then comes the SOC frameworks such as NIST cybersecurity framework. MITRE ATT&CK.
  NIST framework focuses on the lifecycle identify, protect, detect, respond, and recover.
  MITRE is based on real world observations, and it's an globally accessible knowledge base of adversary tactics and techniques.

2. Security Monitoring Basics:

- **Monitoring** is the **act** of **maintaining situational awareness**. Meaning if you did not log it, it never happened.
- Key concepts include **SIEM security information** and **event management**; which is the **central brain**. It collects logs from everywhere, correlates them and alerts the team. Second concept is the **False Positive** struggle; this means the high-quality monitoring tuning; meaning for instance if your **SIEM** fires

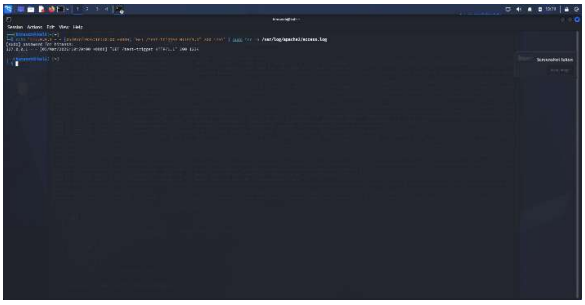
100 alerts and 999 are harmless, your **analysts** will suffer from alert fatigue and miss the 1 real attack.

3. Log Management Fundamentals:

- **Logs** are the **footprints** left behind by **every action** on a **computer**. **Log management** ensures these **footprints** are **readable** and **stored safely**.
- There is log **lifecycle**: which is gather raw data, **normalize** it by **converting** different formats into one, and lastly **storage** keeping logs long enough for compliance.
- Common log sources are **Windows event ID 4625** which is for **failed login**, **windows event ID 4624** which is for **successful login**.

4. Practice Tasks: Log management:

- For this task I was required to setup Fluentd on my kali linux VM to collect syslog. then test it by generating logs with logger “test message’. and then forward the logs to SIEM and verify the receipt.
- Also to add the KQL query: source = "security-login-*" EventID = 4625 | stats count by SourceIP.
- So as required I followed as to do so I installed Logstash first
  by sudo apt install logstash -y
  then, created a mock apache log
  created the directory and file and added a sample log in it:
  sudo mkdir -p /var/log/apache2
  sudo touch /var/log/apache2/access.log
  echo '192.168.1.45 - - [05/Mar/2026:10:15:00 +0000] "GET /admin HTTP/1.1" 403 5034' | sudo tee -a /var/log/apache2/access.log

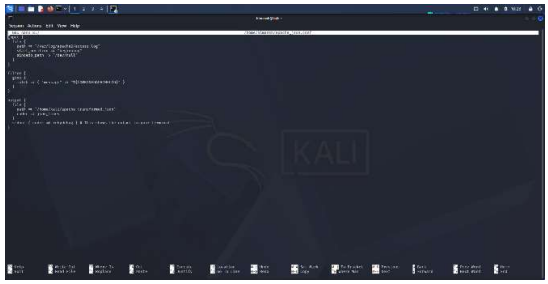


Then to build the **logstash** pipeline:

Firstly Create the **configuration** file

Use the logic I wrote:

```
input {
  file {
    path => "/var/log/apache2/access.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }}
filter {
  grok {
    match => { "message" => "%{COMBINEDAPACHELOG}" }
  }}
output {
  file {
    path => "/home/kali/apache_transformed.json"
    codec => json_lines
  }
  stdout { codec => rubydebug } # This shows the output in your terminal
}
```



then **run Logstash**

sudo /usr/share/logstash/bin/logstash -f ~/apache_json.conf

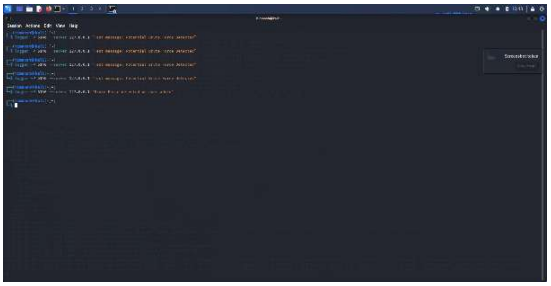
As on successful completion of these

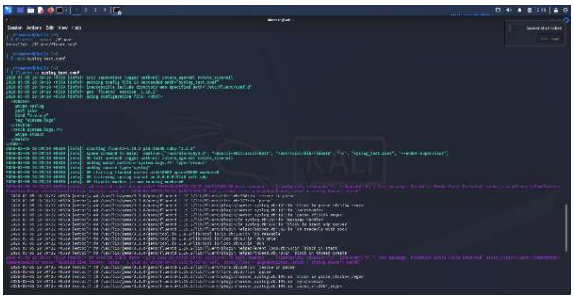
You can open a new terminal and check .json file

cat ~/apache_transformed.json

it will show the logs.

The following shows the Fluentd process:





5. Snort Rule Testing:

- I was **provided** this **rule** to detect **HTTP requests** to **malicious.com**:

  alert tcp any any -> any 80 (msg:"Malicious Domain";

  content:"malicious.com; http_uri; sid:1000001;)

- Since I already had **snort installed** and **ready** to **use**; I opened the local rules file sudo nano /etc/snort/rules/local.rules

  pasted this rule at the bottom:

  alert tcp any any -> any 80 (msg:"Malicious Domain";

  content:"malicious.com; http_uri; sid:1000001;)

- Then **ran**:

  sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -i eth0 -A alert_fast

  triggered curl http://malicious.com in a new terminal

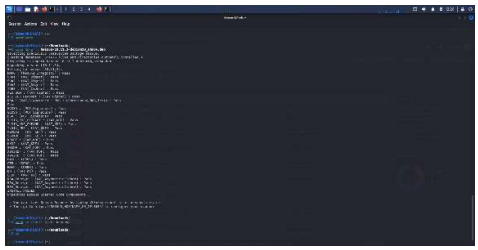
6. Nessus Scan:

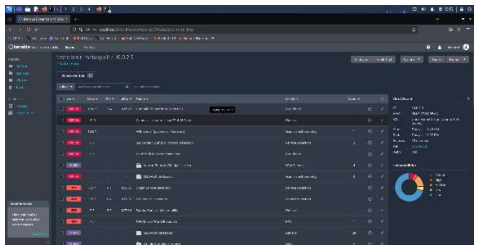
- Firstly, I had to **download Nessus** on my **kali VM** so I **downloaded** the file from the official **Nessus** download page.
  opened terminal in the **downloads folder** and **ran** sudo dpkg -I nessusfile

  Then, **started** the **service**,
  sudo systemctl start nessusd

  then to access the UI I opened the Firefox and went to https://localhost:8834.
  then chose **Nessus Essentials**.
  got activation code,
  created the admin
  and waited for **initialization** to **complete** which took about 10 minutes to **download** and **compile plugins**.

  then clicked on **create a new scan > selected Basic network scan > then entered my Metasploiatble2's IP address which was 10.0.2.3** and then **launched** the test to **run**.





According to the **CVSS** Score the top 3 vulnerabilities were:

1. UnreallRCd Backdoor detection
2. Canonical Ubuntu Linux SEoL(8.04.x)
3. VNC server ‘password’ Password

I couldn’t export the report as it required an upgraded version to do so. it required to upgrade to Nessus professional version.

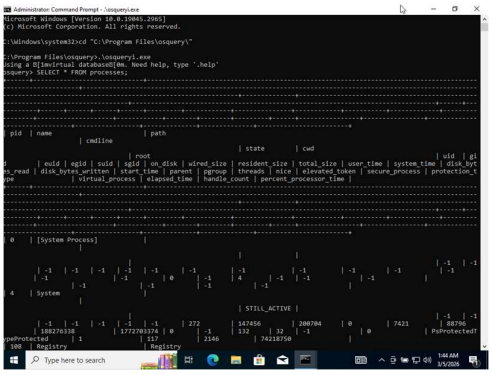
7. Osquery Monitoring:

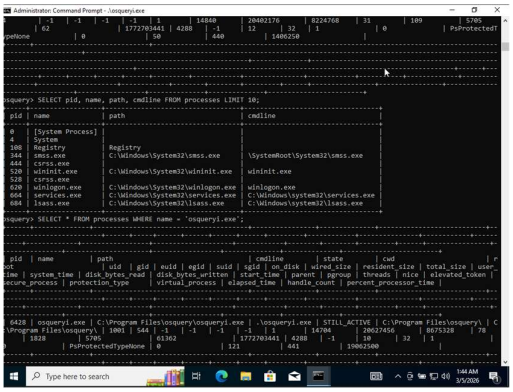
- This particular task was really one my greatest experience so far as I had to **create** a **windows** VM in my **virtual** box, then install **osquery msi** file and **install** it on the **windows vm**, then **write** some **osqueries** which were similar **format** to sql language.
- When I installed **os** query I then **opened command prompt** as **administrator**, **typed**: osqueryi

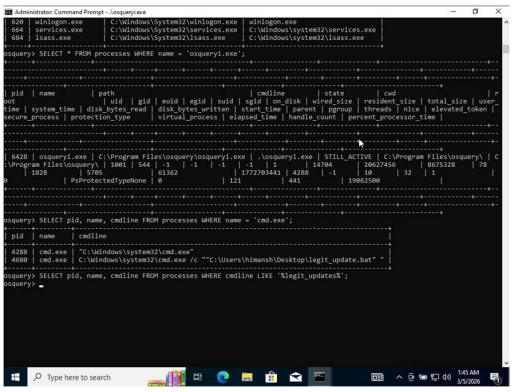
  then SELECT * FROM processes;

  SELECT pid, name, path, cmdline FROM processes LIMIT 10;

  SELECT * FROM processes WHERE name = 'osqueryi.exe';







after this I then wrote a **small malware using** the **notepad** and **saving** it as a **bat** file:

This is the script of the **malware**

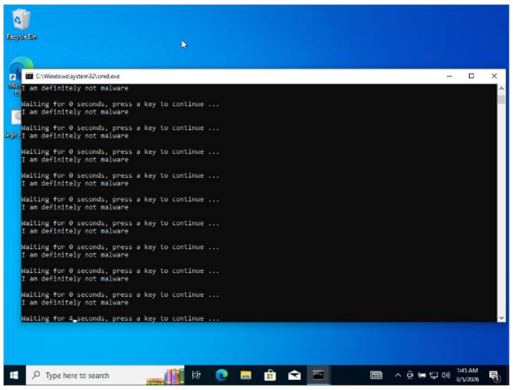
@echo off

:loop

echo I am definitely not malware...

timeout /t 5

goto loop.



then to **threat hunt** I went back to **osqueryi window** to find the **process** by **name**:

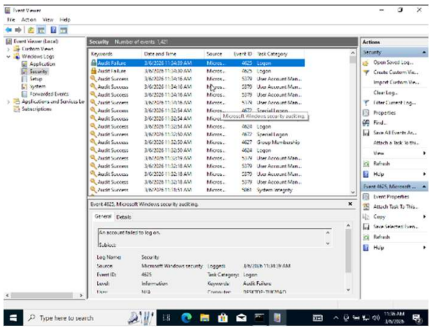
SELECT pid, name, cmdline FROM processes WHERE name = 'cmd.exe';

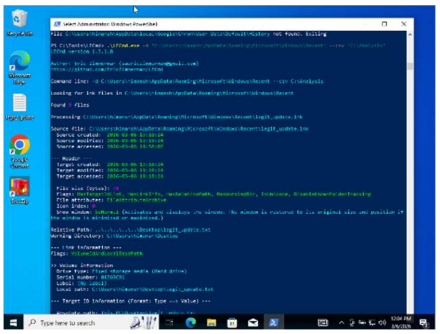
with that it was a successful attempt of using **osquery monitoring** and **simulating** a **malicious process** with a **harmless** batch file.

8. Practical Application:

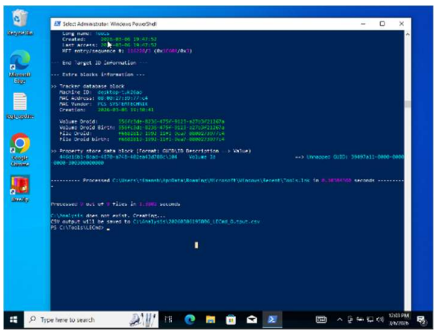
- The **first task** in this section required to use my **Windows VM's Event viewer** to check for **Event ID 4625** which stands for **failed login** which I found in my **windows VM**. Also identified **bruteforce attack** from **security logs**.
- I **simulated** a **bruteforce** attack from my **Kali VM** to **Windows VM** by disabling windows **firewall**, **enabling printer and file sharing**, **enabling SMB public**, to get access to the **VM** through my **Kali linux VM** the **attacker machine**. And also **enabling audit logon service success** or **failure** option in **secpol.msc** from **win+r**.
- Also installed **Zimmerman's tools** on the **windows VM** to **parse chrome history** for **malicious URL's** and get a **csv file** with logs from it.

Audit Logon failure event viewer results:

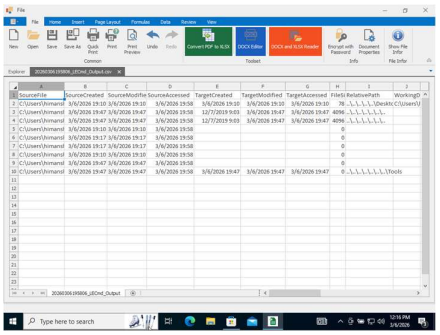




CSV file Creation:



CSV file:



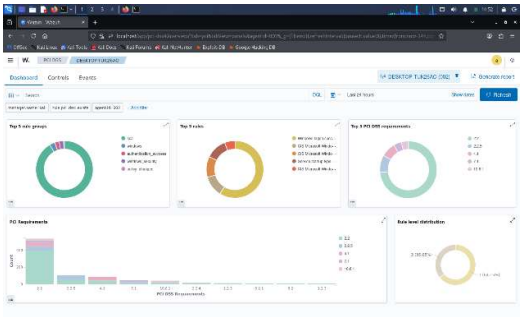
- **This is the Sample document format with fields I created with the results I got from my experience:**

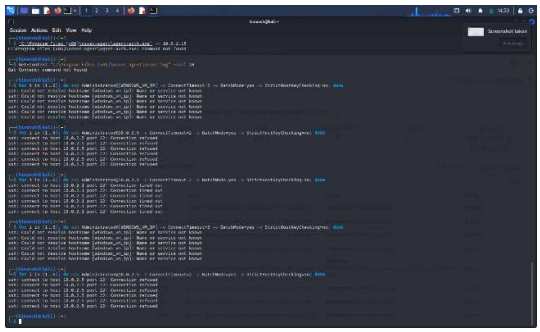
Date/Time	Source IP	Event ID	Description	Action Taken
3/6/2026 11:39:59 AM	10.0.2.15	4625	Multiple failed login attempts detected against the administrator account via SMB.	Isolated the Windows VM from the network and exported logs for forensic review.

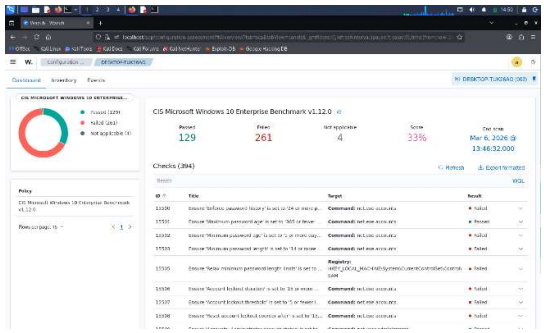
9. Setting up monitoring dashboards using Wazuh:

- I **installed Wazuh** on **kali linux** and **logged** on to the **Dashboard UI** using **Localhost:5601/.**
- Then, connected using add agent and using the command provided by **Wazuh** and entering the command in **powershell** in **windows VM** and gaining a **successful connection** when it showed **Active** on the **Wazuh Dashboard under agent's section**.
- Then, I ran **6 SSH attempts**, which the **manager successfully aggregated** those **events** and **upgraded** the **alert level** to **10** which means **highest**.
- I then used the **Wazuh precreated dashboards** to **visualize** the **results** and **logs** through **piecharts**, **tables**, **bar charts** etc.

Wazuh Prebuilt Nist dashboard format:







- In the **Wazuh** dashboard, we can see the **failed** attempts and **successful attempts** of the **related fields**.